CoachingStudio Functional Specification + Developer PRD — Epic E1: App Skeleton + Mobile Wrapper

Document Name: CoachingStudio_FS_PRD_E1
Version: 1.0
Date: March 30, 2026
Scope: EPIC E1 only
Prerequisite assumption: Epics T0, T1, T2, T3, T4, and E0 are complete, meaning the project foundation, frontend architecture, backend setup, security baseline, deployment/devops baseline, and the public marketing website already exist.[1][2]

1. Purpose

Epic E1 defines the first authenticated application shell for CoachingStudio and packages that shell for mobile delivery using a wrapper approach. The backlog describes E1 as “App Skeleton + Mobile Wrapper,” covering layout, placeholders, routing, menu roles, dashboards, mobile UI, Capacitor, mobile navigation, build, and environment configuration.[1]

The goal of this epic is not to deliver complete business functionality. The goal is to establish a production-grade application container that sits on top of the CoachingStudio web application architecture, supports future role-based features, and can be built as a mobile app wrapper for iOS and Android using Capacitor, which is already part of the technical architecture.[2]

2. Product Context

CoachingStudio is architected as a multi-tenant SaaS platform serving Portal Admins, Coaching Companies, Independent Coaches, and Individual Coachees, with major platform areas including programs, tools, cohorts, assignments, events, reports, and notifications.[3][2]

The technical architecture defines Next.js App Router for the UI, TypeScript for application code, Tailwind plus shadcn/ui for the design system, Zustand for client state where needed, Firebase for auth and data services, and Capacitor as the mobile wrapper layer.[2]

The product document also states that the platform must support a responsive web portal and a wrapper-based native mobile application for iOS and Android, with mobile access being an engagement multiplier for assignments, tools, dashboards, and event participation.[3]

3. Epic Intent

E1 is the bridge between the public website and the fully functional logged-in product. It creates the shell into which future epics such as authentication, dashboards, admin management, programs, tools, cohorts, assignments, events, notifications, and reports will plug.[1]

This means the deliverable for E1 is a navigable application framework with role-aware structure, placeholder screens for future modules, mobile-first interaction rules, Capacitor packaging readiness, and a reproducible build configuration for web and mobile targets.[1][2]

4. Scope

This epic includes the following user stories from the backlog:[1]

- US-E1-01: Layout
- US-E1-02: Placeholders
- US-E1-03: Routing
- US-E1-04: Menu Roles
- US-E1-05: Dashboards

- US-E1-06: Mobile UI
- US-E1-07: Capacitor
- US-E1-08: Mobile Nav
- US-E1-09: Build
- US-E1-10: Env Config

5. Out of Scope

The following are explicitly out of scope for E1:

- Real authentication flows and identity lifecycle, which belong to E2.[1]
- Live role assignment, approval, onboarding, and profile management.[1]
- Live dashboard data, metrics, charts, and assignment lists from Firestore.[4][1]
- Functional programs, tools, reports, cohorts, assignments, events, notifications, and admin CRUD flows.[1][3]
- Native push notification enablement and advanced mobile-native capabilities, which sit later in the roadmap and mobile enhancement epic.[1][2]
- Offline-first data synchronization beyond simple wrapper behavior.

6. Epic Outcome

At the end of E1, the developer must deliver a working app shell that includes:

- A signed-in application layout distinct from the public landing page.
- Placeholder routes/screens for core future platform modules.
- Role-sensitive menus for Admin, Coaching Company, Coach, and Individual personas.[3][2]
- Dashboard landing pages for each role with placeholder widgets/cards.
- A mobile-optimized version of the app shell.
- Capacitor integration enabling Android and iOS wrapper packaging.[2]

- Build and environment conventions so the same codebase can target web and mobile reliably.[2][1]

7. User Segments and Role Model

The platform has four actor types in product strategy and architecture: Portal Admin, Coaching Company, Independent Coach, and Individual Coachee.[3][2]

For E1, those actors translate into four role-aware shell experiences:

Role	Shell purpose	Dashboard landing target
Portal Admin	Platform governance and system oversight shell	Admin dashboard placeholder [3][1]
Coaching Company	Multi-coach tenant shell	Company dashboard placeholder [3]
Independent Coach	Solopreneur coaching workspace shell	Coach dashboard placeholder [3][1]
Individual	Personal growth workspace shell	Individual dashboard placeholder [3][1]

The role model must align with the security architecture, which uses user context and RBAC to drive access control and downstream visibility.[2][4]

8. Relationship with Epic E0

Epic E0 delivered the public-facing marketing site as a single-page responsive experience with SEO, analytics, public navigation, and chatbot launcher behavior.[1]

Epic E1 does not replace E0. Instead, E1 introduces the authenticated product shell that the E0 site eventually routes users into through Sign In and Register entry points. In mobile-wrapper terms, E1 is the first stage where CoachingStudio becomes an installable app experience rather than only a public website.

9. Experience Principles

The application shell must follow these principles:

- Fast to understand after login.
- Role-aware but structurally consistent.
- Mobile-first and touch-friendly.
- Safe for incomplete modules by using explicit placeholders.
- Navigationally stable across web and mobile.
- Wrapper-compatible with no assumptions that only desktop browsers are in use.
- Clean separation of UI shell from business logic, consistent with the architecture rule that UI should not contain backend business logic and should not access Firestore directly.[2]

10. Information Architecture for E1

The app shell must establish the top-level product areas that later epics will implement. The route map should reflect the major product domains already present in the product, technical, and backlog documents.[2][1][3]

Minimum authenticated route groups for E1:

- Dashboard
- Programs
- Tools
- Events
- Cohorts
- Assignments
- Reports
- Notifications
- Settings / Profile shell
- Admin-only management shell

Because many of these features are not yet implemented, E1 may expose them as placeholder screens. The purpose is to normalize the application skeleton and route hierarchy before later feature work begins.[1]

11. Functional Specification by User Story

11.1 US-E1-01 Layout

Objective

Create the primary authenticated application layout for CoachingStudio.[1]

Core requirement

The shell must be visually and structurally different from the public marketing site created in E0. The app layout is a product workspace, not a landing page.

Layout regions

Desktop/tablet app layout must include:

- App header.
- Left sidebar navigation.
- Main content area.
- Optional contextual right rail not required in E1.
- Global feedback area for loading, empty, and placeholder states.

Mobile app layout must include:

- Top app bar.
- Primary content area.
- Mobile navigation pattern as defined in US-E1-08.

Header requirements

App header must include:

- CoachingStudio wordmark/logo.
- Current workspace/page title.
- Role badge or role label.
- Optional profile/avatar trigger area.
- Optional notification bell placeholder.

Sidebar requirements

Desktop sidebar must support:

- Logo/app identity at top.
- Role-specific menu items.
- Clear active state.
- Collapsible mode optional but recommended.
- Footer zone for settings and sign-out placeholder.

Layout sizing guidance

Desktop:

- Header height: 64 px to 72 px.
- Sidebar width expanded: 240 px to 280 px.
- Content max width: fluid; use full app width with internal section constraints.

Tablet:

- Sidebar may collapse to icon rail or drawer.

Mobile:

- No persistent desktop sidebar.
- Bottom nav or drawer-based primary navigation.

Layout behavior

- The app shell header remains sticky.
- Main content is independently scrollable.
- The shell must avoid dual-scroll confusion as much as possible.
- Safe areas must be respected on mobile devices with notches and gesture bars.

11.2 US-E1-02 Placeholders

Objective

Provide developer-ready placeholder screens and states for all future modules in scope of near-term epics.[1]

Placeholder philosophy

Because E1 creates the shell before E2+ business modules are fully implemented, placeholders must be deliberate, not blank pages. Each placeholder page should clearly communicate:

- Module name.
- Intended purpose.
- Future user value.
- Status label such as “Coming in next epic” or “Module shell ready”.
- Optional CTA back to dashboard.

Required placeholder routes

At minimum, placeholders must exist for:

- Dashboard variants.
- Programs.
- Tools.
- Events.
- Cohorts.
- Assignments.
- Reports.
- Notifications.
- Settings/Profile.
- Admin management modules.

Placeholder component structure

Each placeholder page should contain:

- H1 title.
- One-line description.
- Status badge.
- Placeholder card grid or skeleton blocks representing future content.
- Optional illustrative icon.

Placeholder standards

- No dead-end empty white screens.
- No “404” behavior for not-yet-built internal routes.
- Every linked route from menus must resolve to a stable screen in E1.

11.3 US-E1-03 Routing

Objective

Define the foundational authenticated route structure for CoachingStudio.[1]

Routing requirements

The technical architecture uses Next.js App Router and already defines `/src/app` as the routing layer.[2]

The developer must implement an authenticated route group such as:

```
/src/app/
  (public)/
    page.tsx
  (app)/
    dashboard/page.tsx
    programs/page.tsx
    tools/page.tsx
    events/page.tsx
    cohorts/page.tsx
    assignments/page.tsx
    reports/page.tsx
    notifications/page.tsx
    settings/page.tsx
    admin/
      users/page.tsx
      programs/page.tsx
      tools/page.tsx
```

Route rules

- Public marketing route from E0 remains on the public side.[1]
- Authenticated application lives under a dedicated route group/layout.
- Route naming should be stable and feature-oriented.
- Future feature stories should plug into existing routes instead of redefining route hierarchy.

Auth guard handling in E1

Because E2 owns auth, E1 may use a mock session or feature-flag-based dev shell access. However, the route structure must be guard-ready so later auth guards can be inserted with minimal refactoring, aligning with the earlier security baseline and future auth work.[1][2]

Role routing behavior

A role-aware dashboard redirect should be supported conceptually:

- Admin → `/dashboard/admin`
- Coaching Company → `/dashboard/company`
- Coach → `/dashboard/coach`
- Individual → `/dashboard/individual`

If developer simplicity is preferred, a single `/dashboard` route may render role-specific dashboard content based on a mock `userContext`, which aligns with the broader user-context architecture.[2][4]

11.4 US-E1-04 Menu Roles

Objective

Define role-based menu visibility in the app shell.[1]

Role model source

The role and access model must align with the actor model and RBAC architecture already defined in the strategy, technical, and database documents.[3][2][4]

Menu behavior

Menu items must be filtered by role. In E1 this can be driven by mock role data, but the menu contract must be ready for actual userContext integration later.[2][4]

Recommended role-to-menu matrix

Menu Item	Admin	Coaching Company	Coach	Individual
Dashboard	Yes	Yes	Yes	Yes
Programs	Yes	Yes	Yes	Yes
Tools	Yes	Yes	Yes	Yes
Events	Yes	Yes	Yes	Yes
Cohorts	Optional	Yes	Yes	No
Assignments	Yes	Yes	Yes	Limited/My items
Reports	Yes	Yes	Yes	Own reports only
Notifications	Yes	Yes	Yes	Yes
Users/Admin Console	Yes	Limited company admin area	No	No
Settings/Profile	Yes	Yes	Yes	Yes

This role partition reflects the platform's multi-tenant and role-based access structure, where not all actors have visibility into the same management functions.[3][2][4]

Menu data model

Menu configuration should be centralized in a typed configuration file with:

- label
- route
- icon
- rolesAllowed
- mobileVisible
- desktopVisible
- futureFeatureFlag optional

11.5 US-E1-05 Dashboards

Objective

Provide dashboard landing shells for each main role.[1]

Dashboard purpose

The product document explicitly describes differentiated dashboard needs, including platform analytics for admins, organizational oversight for coaching companies, coaching delivery views for coaches, and personal growth dashboards for individuals.[3]

Required role dashboards

- Admin dashboard placeholder.
- Coaching company dashboard placeholder.
- Coach dashboard placeholder.
- Individual dashboard placeholder.

Dashboard placeholder structure

Each dashboard must include:

- Personalized heading.
- Summary cards or KPI placeholders.
- Quick-action cards.
- Recent activity placeholder list.
- Module shortcuts relevant to the role.

Example dashboard expectations

Admin dashboard placeholder may include cards for:

- Users
- Programs
- Tools
- Platform metrics
- Events
- Notifications

Company dashboard placeholder may include cards for:

- Coaches
- Cohorts
- Program usage
- Tool engagement
- Upcoming events

Coach dashboard placeholder may include cards for:

- Active coachees
- Active cohorts
- Pending assignments
- Programs in progress
- Upcoming events
- Latest reports

Individual dashboard placeholder may include cards for:

- My assignments
- My programs
- My tools
- My reports
- Upcoming events
- Growth journey progress

These structures follow the role expectations described in the product document and later backlog epics.[3][1]

11.6 US-E1-06 Mobile UI

Objective

Ensure the app shell is mobile-ready and behaves like a mobile-first application, not just a shrunk desktop web app.[1][3]

Mobile UI rules

- Primary interactions must be reachable by thumb.
- Navigation must not rely on hover.
- Cards, lists, and tiles must stack vertically.
- Forms and buttons must use touch-friendly sizes, minimum 44 px hit area.
- Use device safe-area insets for top/bottom bars.
- All primary shell actions must remain visible without desktop assumptions.

Mobile content rules

- Dashboard widgets stack into one column.
- Placeholder modules use vertical card layouts.
- Long tables are not required in E1; use cards/placeholders instead.
- Motion should be lightweight and native-feeling.

Mobile shell states

The shell must be tested in portrait orientation as the primary experience. Landscape may be supported passively, but design optimization is for portrait use, consistent with earlier mobile-first guidance from E0 and the product's mobile-wrapper direction.[3]

11.7 US-E1-07 Capacitor

Objective

Integrate Capacitor so the app shell can be packaged as an Android and iOS wrapper application.[1][2]

Architecture alignment

Capacitor is explicitly named in the technical architecture as the mobile wrapper technology for CoachingStudio.[2]

Functional requirements

- Capacitor project must be initialized in the repository.
- Web build output must sync into Capacitor correctly.
- Android project scaffolding must be created.
- iOS project scaffolding must be created, even if iOS build is not run in the initial dev environment.
- App metadata such as app name, app ID, and icons/splash placeholders must be configurable.

Initial plugin scope

Only core wrapper setup is required in E1. Native plugins may be added later. E1 should not over-commit to push notifications, camera, files, or advanced deep linking unless required for baseline wrapper operation.

Wrapper behavior requirements

- App launches to CoachingStudio app shell, not the public landing page.
- Navigation inside wrapper stays in-app.
- External links must open appropriately when explicitly intended.
- Keyboard and viewport resizing should not break form or nav layouts.

11.8 US-E1-08 Mobile Nav

Objective

Define mobile-first navigation for the app shell.[1]

Mobile nav options

For E1, the preferred pattern is a bottom navigation bar for the most important destinations, plus an overflow drawer for secondary items.

Recommended bottom-nav items

- Home/Dashboard
- Programs
- Tools
- Events
- More

If role requires other modules, the “More” item opens a sheet/drawer with additional destinations such as cohorts, assignments, reports, notifications, admin, and settings.

Mobile nav rules

- Bottom nav always visible on mobile authenticated screens unless a full-screen modal is open.
- Active route clearly indicated.
- Icons plus short labels.
- Maximum 5 bottom nav items.
- Overflow items accessible within 1 tap from the bottom bar.

Role-aware mobile nav

Menu composition must remain role-aware. For example, an individual user should not see cohorts or admin links in the bottom-nav overflow if those functions are unavailable in the role model.[3][4]

11.9 US-E1-09 Build

Objective

Establish reproducible build behavior for web and mobile wrapper outputs.[1]

Build outputs required

- Web application build for Next.js.
- Static/exportable or wrapper-compatible build artifact for Capacitor consumption, depending on chosen integration approach.
- Android build readiness.
- iOS build readiness.

Developer build requirements

At minimum, the developer must document and validate:

- Local web dev command.
- Production web build command.
- Wrapper sync command.
- Android run/build command.
- iOS sync/build command.

Recommended build pipeline

A practical E1 build pipeline is:

1. Run Next.js web build.
2. Produce web output compatible with Capacitor webDir strategy.
3. Sync Capacitor assets.
4. Open/run native shells.

Build quality gates

- No TypeScript compile errors.
- No unresolved route references.
- No menu links pointing to missing pages.
- No environment variable failures on mobile startup.
- Capacitor sync completes successfully.

11.10 US-E1-10 Env Config

Objective

Define the environment configuration required for the app shell across dev, staging, prod, and mobile-wrapper targets.[1][2]

Environment baseline

The technical architecture already requires separate dev, staging, and prod environments across Vercel and Firebase projects.[2]

Required env categories

- Public frontend env values.
- Firebase config values appropriate for frontend initialization.
- App display metadata values for wrapper build.
- Analytics toggles if relevant.
- Feature flags for mock auth / placeholder mode in E1.

Env rules

- No secrets in frontend code.[2]
- Secrets for backend integrations remain in Firebase Functions or server-side secure configuration, not in the app shell.[2]
- Public and private env variables must be clearly separated by naming convention.
- Mobile wrapper builds must read from the same environment strategy without hardcoding environment-specific URLs into components.

Recommended env variables

Examples include:

- `NEXT_PUBLIC_APP_ENV`
- `NEXT_PUBLIC_APP_BASE_URL`
- `NEXT_PUBLIC_FIREBASE_API_KEY`
- `NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN`
- `NEXT_PUBLIC_FIREBASE_PROJECT_ID`
- `NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET`
- `NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID`
- `NEXT_PUBLIC_FIREBASE_APP_ID`
- `NEXT_PUBLIC_ENABLE_MOCK_SESSION`
- `NEXT_PUBLIC_DEFAULT_ROLE`

These are examples only; exact naming should remain aligned with the repository conventions established in T0–T3.[1][2]

12. Detailed App Layout Specification

12.1 Desktop layout

Desktop app shell should use a two-region structure:

- Left sidebar for navigation.
- Right content region containing sticky top bar and page content.

Recommended frame:

- Sidebar: 256 px.
- Header: 68 px.
- Content region padding: 24 px to 32 px.
- Content cards/grid spacing: 16 px to 24 px.

12.2 Tablet layout

Tablet shell may use:

- Collapsed side rail with icons only, or
- Drawer navigation triggered from app bar.

The choice must remain consistent across all app screens.

12.3 Mobile layout

Mobile shell should use:

- Top app bar with logo, page title, and action icon area.
- Main content area with 16 px horizontal padding.
- Bottom navigation with 4 primary destinations plus “More”.
- Optional bottom sheet for overflow navigation.

12.4 Visual style

The app shell should inherit brand continuity from E0 but use a more product-oriented interface language:

- Cleaner surfaces.
- More structured information density.
- Smaller typography than the landing page.
- Strong active-state navigation cues.

- Card-based dashboard scaffolding.

13. Placeholder Content Strategy

Each placeholder screen must make future development easier by implying the eventual data model and UI density. For example:

- Programs placeholder should preview catalog/list cards because programs and modules already exist in the data model.[4][3]
- Tools placeholder should preview assessment cards and pending tool actions because tools, submissions, and results are already core data concepts.[4][3]
- Assignments placeholder should preview the assignment-driven architecture, which is explicitly central to the technical and database design.[2][4]
- Reports placeholder should preview generated report items because reports are a first-class platform output.[4][3]

This keeps the shell semantically aligned with actual future features instead of using generic lorem ipsum placeholders.

14. Technical Specification

14.1 Stack alignment

The implementation must use the architecture already defined for CoachingStudio:[2]

- Next.js App Router.
- TypeScript.
- Tailwind + shadcn/ui.
- Zustand only where shared shell state is needed.
- Firebase client integration patterns through service layers, not direct UI DB access.
- Capacitor as mobile wrapper.

14.2 Engineering rules

Non-negotiable engineering rules inherited from the architecture:[2]

- No direct database calls from UI.
- No business logic in UI components.
- All backend business logic belongs in Functions.
- Role-aware shell should be built to consume `userContext` later.
- Keep modules isolated and consistent with module structure.

14.3 Recommended file structure

```
/src/app/
  (public)/
    page.tsx
  (app)/
    layout.tsx
    dashboard/page.tsx
    programs/page.tsx
    tools/page.tsx
    events/page.tsx
    cohorts/page.tsx
    assignments/page.tsx
    reports/page.tsx
    notifications/page.tsx
    settings/page.tsx
    admin/users/page.tsx
    admin/programs/page.tsx
    admin/tools/page.tsx
/src/modules/app-shell/
  components/
    app-header.tsx
    app-sidebar.tsx
    app-bottom-nav.tsx
    app-more-sheet.tsx
    app-role-switcher-dev.tsx
    placeholder-page.tsx
    dashboard-cards.tsx
  config/
```

```
    app-routes.ts
    menu-by-role.ts
  hooks/
    use-current-role.ts
    use-mobile-nav.ts
  types/
    shell.types.ts
/src/lib/
  capacitor.ts
  env.ts
/src/constants/
  roles.ts
  routes.ts
/capacitor.config.ts
```

14.4 State management

E1 may use local state for many UI interactions, but shared shell state is acceptable for:

- active role in mock/developer mode,
- mobile drawer open/close,
- current nav state,
- app-shell user stub,
- theme mode if used.

A small shell-focused Zustand store is acceptable because Zustand is already part of the technical stack.[2]

14.5 Routing implementation pattern

Use layout nesting in App Router so all authenticated screens share a common shell. This avoids duplicating headers/nav and creates a single injection point for future auth guards.

Recommended pattern:

- Public layout for E0.
- App layout for E1 shell.
- Nested pages for each module.

- Optional middleware or shell guard stub for future E2 integration.

14.6 Placeholder data contracts

Use typed mock data objects for:

- dashboard KPI cards,
- recent activity lists,
- module quick links,
- role-aware labels,
- placeholder page descriptions.

This allows the shell to look intentional while remaining independent of live Firestore integration.

14.7 Capacitor implementation guidance

The developer should configure Capacitor with:

- stable appId,
- appName,
- webDir matching the built web artifact,
- Android and iOS platforms added to the repo,
- optional plugin baseline only if required for shell stability.

The public website and authenticated app shell may coexist in the same Next.js codebase, but the wrapper should deep-link directly into the authenticated shell entry or shell root, depending on app-start strategy.

14.8 Error handling

App shell must provide the following fallback states:

- unknown role fallback,
- missing route fallback,
- placeholder data load fallback,
- mobile-nav open/close failure safe state,
- wrapper startup error message for fatal boot issues.

Next.js route-level error handling should align with the architecture's centralized error philosophy.[2][1]

15. Role-Based Dashboard Requirements

Admin dashboard shell

Must communicate platform oversight consistent with the product's admin responsibilities such as user oversight, global content catalog, global events, analytics, and communications.[3]

Coaching company dashboard shell

Must communicate team-level operational oversight, multi-coach management, analytics, and shared organization delivery.[3]

Coach dashboard shell

Must communicate the coach's working view of coachees, cohorts, tools, assignments, events, and reports, which is central to the independent coach value proposition in the product document.[3]

Individual dashboard shell

Must communicate the personal growth journey: assigned items, self-assessments, programs, reports, and events.[3]

16. Mobile Wrapper-Specific Requirements

Because the product document explicitly positions the mobile application as a wrapper-based native app for iOS and Android, E1 must prepare the shell for wrapper realities rather than browser-only assumptions.[3]

Required wrapper considerations:

- Safe area padding.
- Smooth startup from splash to shell.
- No dependency on browser-only patterns such as hover menus.
- Forms and content must tolerate soft keyboard behavior.
- Internal route transitions must feel app-like and not expose public-web-only assumptions.

- Public landing page should not be the default post-install destination for authenticated/mobile-intent users.

17. Build and Deployment Guidance

The technical architecture already defines GitHub to Vercel auto-deploy for frontend and Firebase CLI for backend, with branch-based environments and rollback capability.[2]

For E1, the build strategy must additionally support wrapper lifecycle operations:

- local dev against web shell,
- staging build for mobile QA,
- production build for store submission preparation later,
- Capacitor sync after web build changes.

The developer should document the commands in the repo README or E1 delivery notes, but the shell code must also be structured so wrapper sync is routine rather than fragile.

18. Acceptance Criteria

US-E1-01 Layout

- Authenticated app shell exists and is distinct from the public E0 site.[1]
- Desktop, tablet, and mobile shell layouts behave as specified.
- Shared shell components are reused across app routes.

US-E1-02 Placeholders

- All menu-linked modules resolve to intentional placeholder pages.[1]
- No dead-end internal routes exist in E1.

US-E1-03 Routing

- App Router structure exists for authenticated shell routes.[2][1]
- Route hierarchy is stable and future-ready.

US-E1-04 Menu Roles

- Role-aware menu visibility works using mock or dev role context.[1][4]
- Users only see role-appropriate menu items.

US-E1-05 Dashboards

- Dashboard placeholders exist for Admin, Coaching Company, Coach, and Individual roles.[1][3]
- Each dashboard has role-relevant widgets and shortcut blocks.

US-E1-06 Mobile UI

- App shell is mobile-friendly, touch-friendly, and portrait-first.[1][3]
- Core flows remain usable at common mobile widths.

US-E1-07 Capacitor

- Capacitor is integrated and shell can be wrapped for Android/iOS.[2][1]
- Wrapper launches into the app shell successfully.

US-E1-08 Mobile Nav

- Bottom navigation or equivalent mobile nav exists and is role-aware.[1]
- Maximum five primary bottom-nav items are used.

US-E1-09 Build

- Web build succeeds.
- Wrapper sync succeeds.
- Native platform scaffolding is in place.[1][2]

US-E1-10 Env Config

- Environment variables required by shell startup are defined and documented.[1][2]
- Public/private env separation is respected.[2]

19. Non-Functional Requirements

- Responsive across modern web browsers and mobile form factors.
- Consistent code organization under the architecture module pattern.[2]
- Type-safe route/menu config using TypeScript.[2]
- No secrets shipped in frontend bundle.[2]
- No Firestore direct access from UI.[2]
- Wrapper-ready without native feature overreach.
- Maintainable placeholder strategy that can be replaced incrementally as E2–E13 are delivered.[1]

20. Developer Notes and Open Inputs

The following product decisions may still need explicit confirmation but should be parameterized in the implementation rather than hardcoded:

- Final app icon and splash assets.
- Final package/app ID.
- Whether the shell launches to a mock dashboard or a login route when auth is introduced.
- Whether coaching company has distinct admin sub-routes in the first iteration.
- Final role names and labels shown in UI.
- Whether bottom nav uses 4 or 5 persistent destinations.
- Whether notifications appear as a top-bar bell or in overflow menu on mobile.

21. Delivery Guidance

Epic E1 should be delivered as durable platform infrastructure, not as a throwaway prototype. The shell must be coded so that later epics can replace placeholder content with real modules while preserving layout, routing, role visibility, and mobile-wrapper behavior.

The most important success criterion is architectural stability: later work on auth, dashboards, programs, tools, events, cohorts,

assignments, reports, notifications, and mobile enhancements should plug into E1 cleanly without major shell redesign.[1][2]